

CS231n 10-12

김동원

CS231n 10-12

김동원

목차

RNN

CNN과 RNN

LSTM

Semantic Segmentation

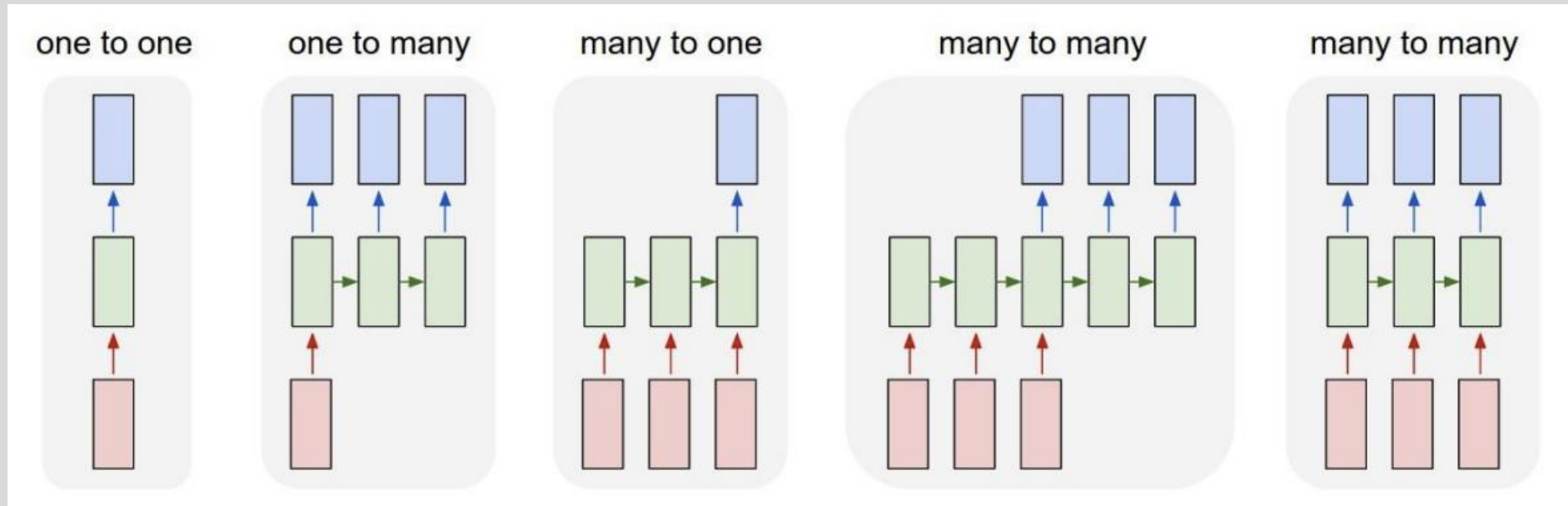
Classification + Localization

Object Detection

CNN의 원리

RNN

학습 데이터크기와 출력이 유연하다



RNN 구조

Vanila RNN

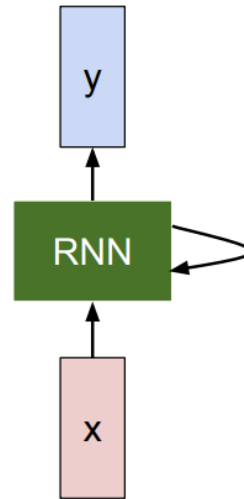
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state some function with parameters W old state input vector at some time step

(Vanilla) Recurrent Neural Network

The state consists of a single “hidden” vector $\mathbf{h}$:



$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

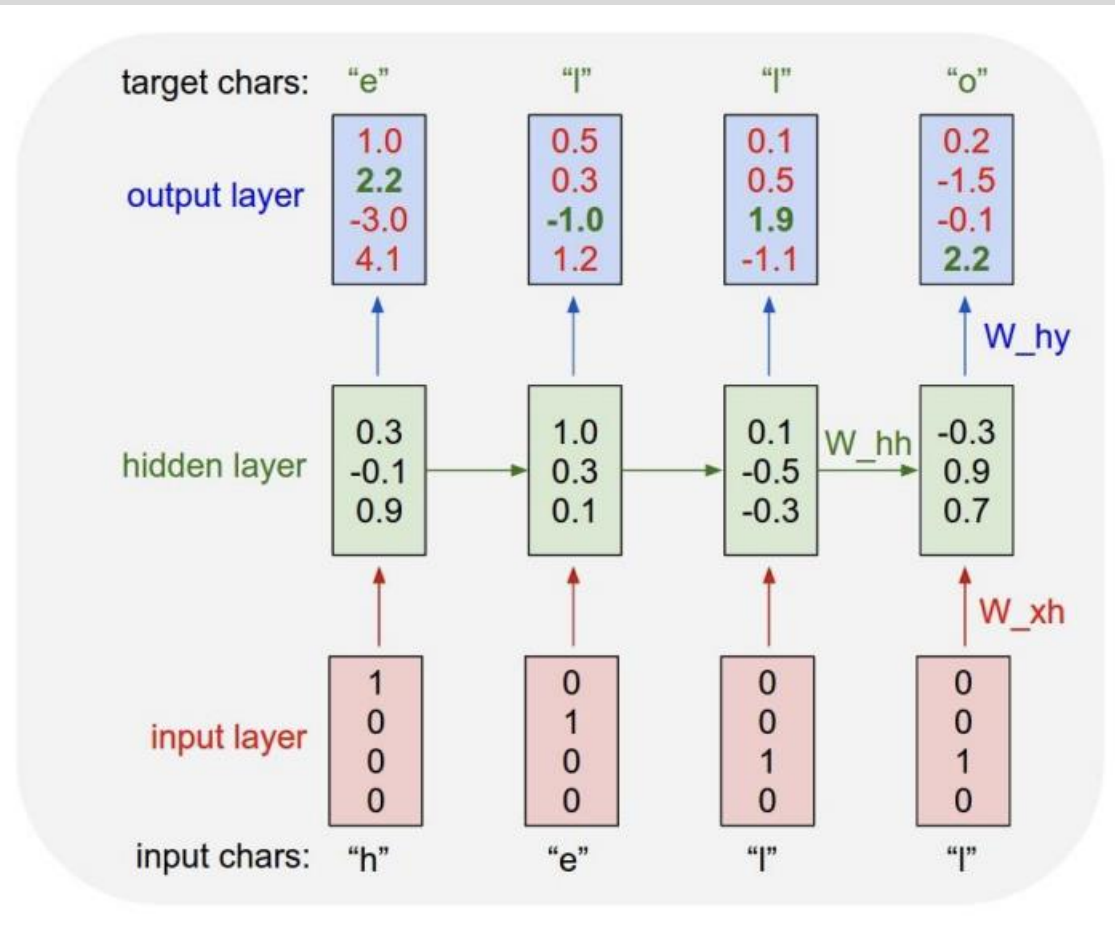
RNN 예시

‘hello’를 각 문자별로 입력하고 다음 문자를 예측하는 모델

Example: Character-level Language Model

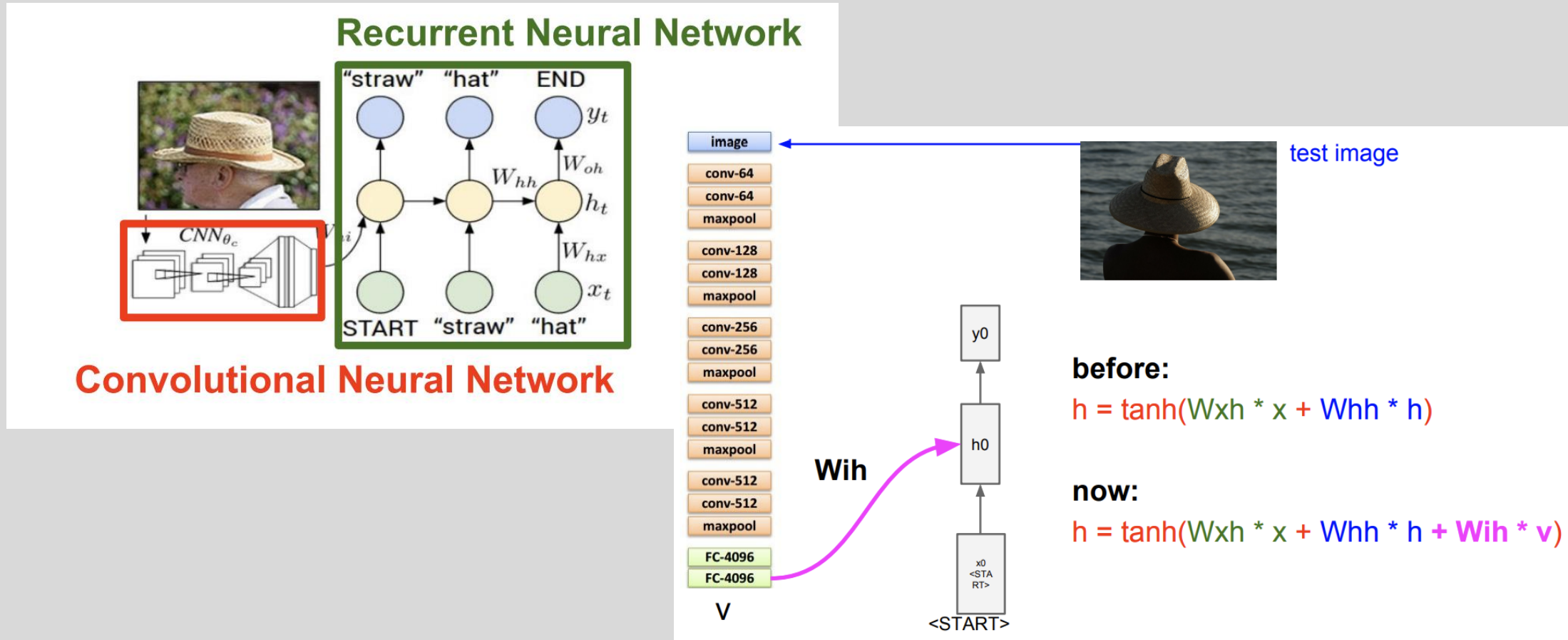
Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”



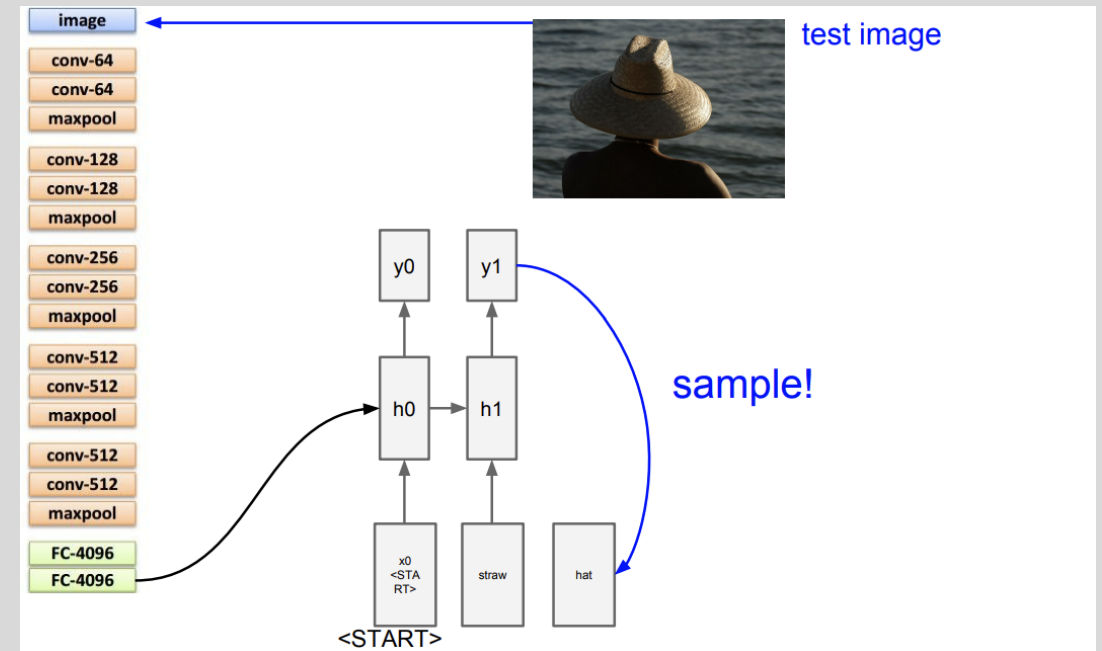
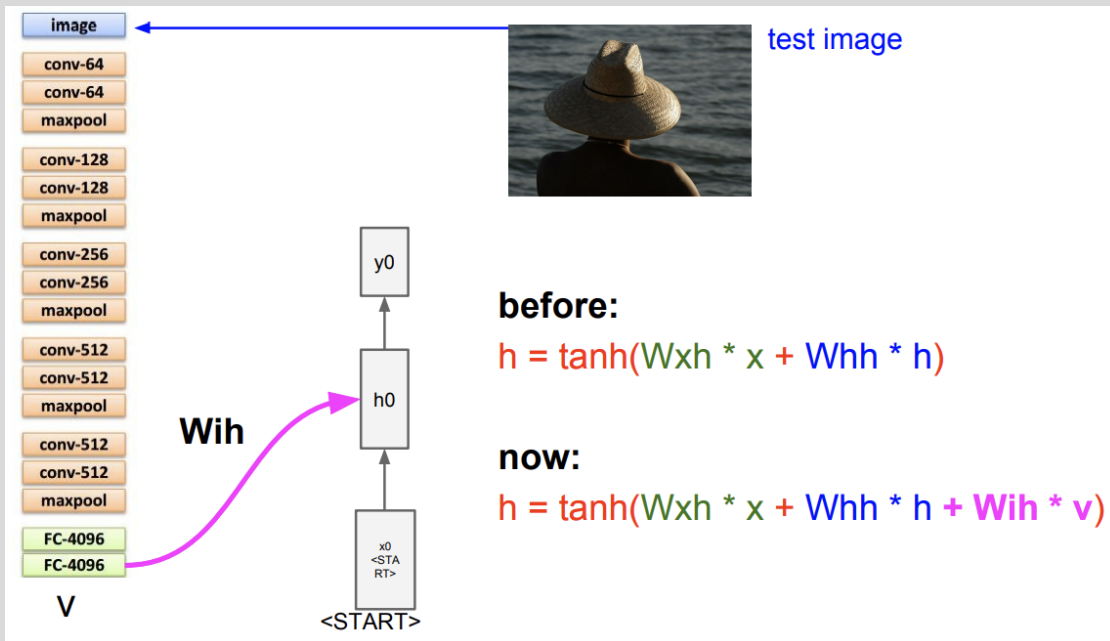
CNN과 RNN

이미지를 설명하는 문장을 만들어내는 모델



CNN과 RNN

이미지를 설명하는 문장을 만들어내는 모델

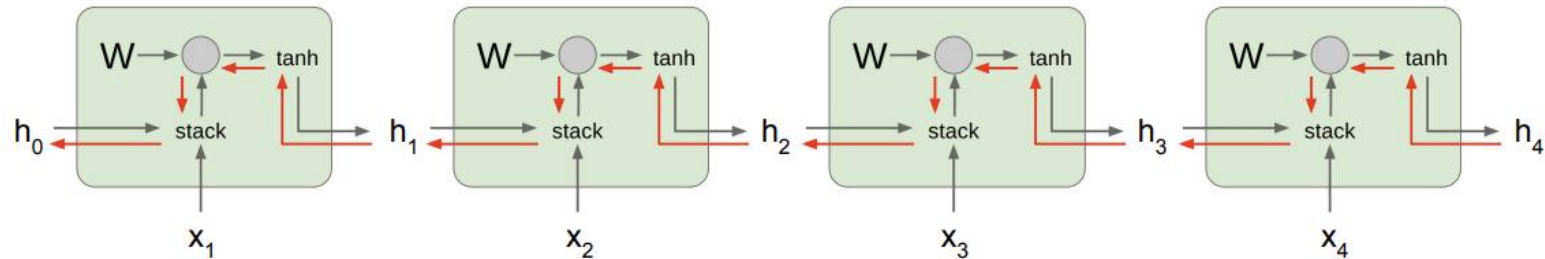


LSTM

RNN의 입력이 길어지면 기울기 계산과정에서 W 가 계속 곱해지며 기울기 소실과 기울기 폭주의 문제점이 발생하여 LSTM이 이를 해결할 수 있다.

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



Computing gradient of h_0 involves many factors of W (and repeated $\tanh$)

Largest singular value > 1 :
Exploding gradients

Largest singular value < 1 :
Vanishing gradients

Gradient clipping: Scale gradient if its norm is too big

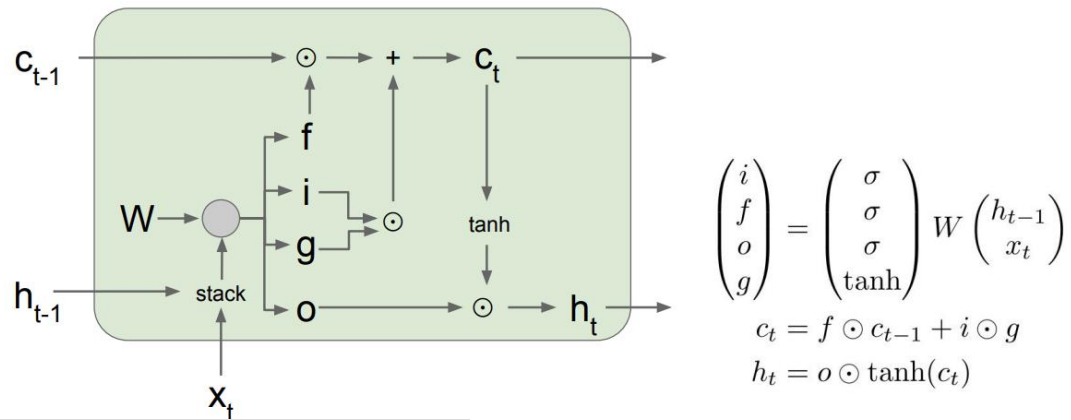
```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

LSTM

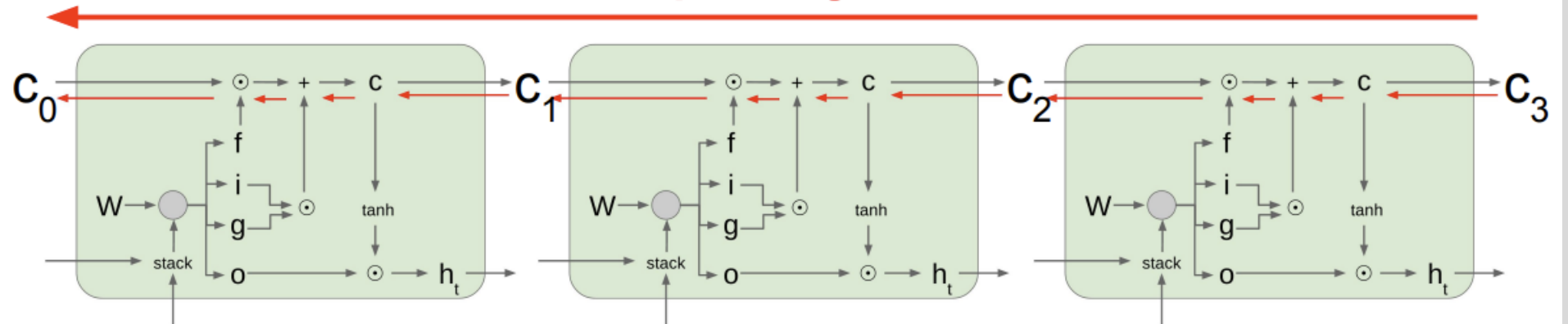
LSTM은 입력이 길어져도 이전 셀과 입력을 조절하여 값을 가져갈 수 있다. 또한 셀 상태를 업데이트 해가는 과정이 개선되어 기울기 소실과 폭주 문제를 막을 수 있다.

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

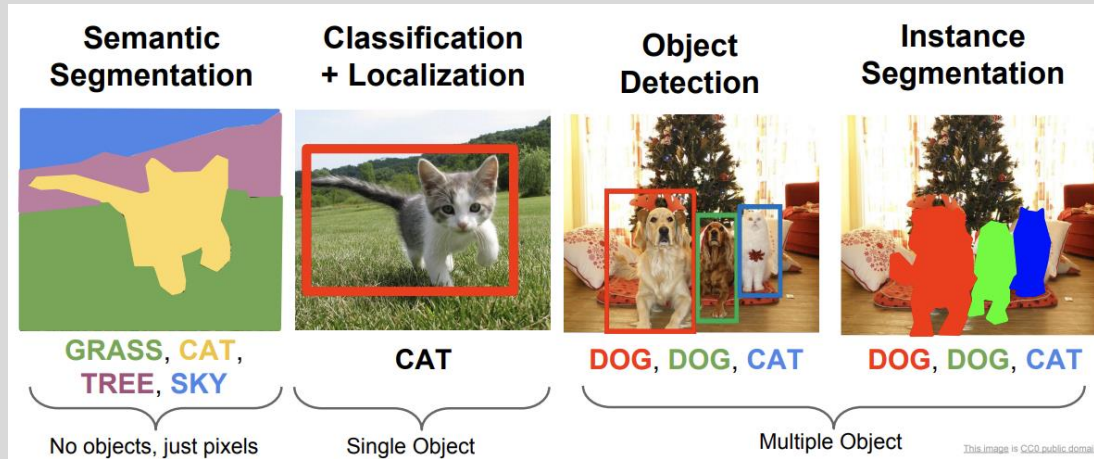


Uninterrupted gradient flow!

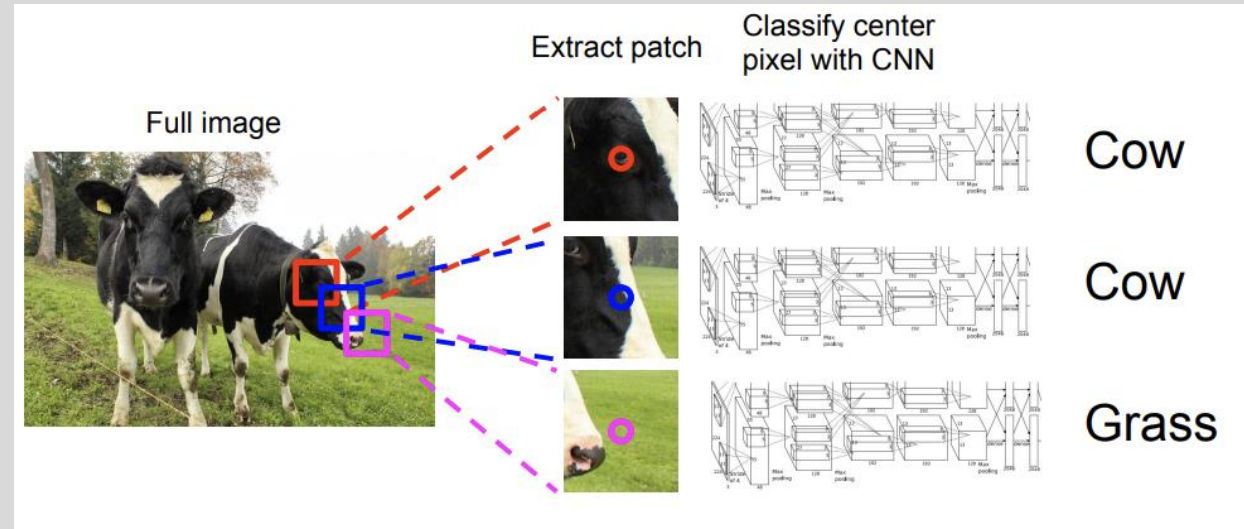


Semantic Segmentation

이미지의 픽셀단위로 물체를 구별한다.



Sliding window방법: 각 픽셀이 중심인 이미지를 CNN모델을 통해 구분한다.

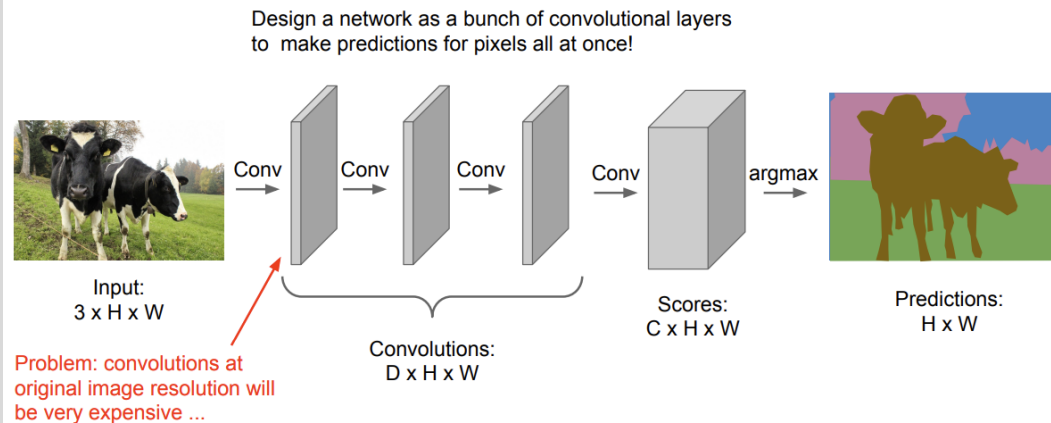


Semantic Segmentation

Fully Convolutional 방법: convolution 레이어로만 이루어진 아키텍처를 마지막 레이어에서 스코어를 통해 구분한다.

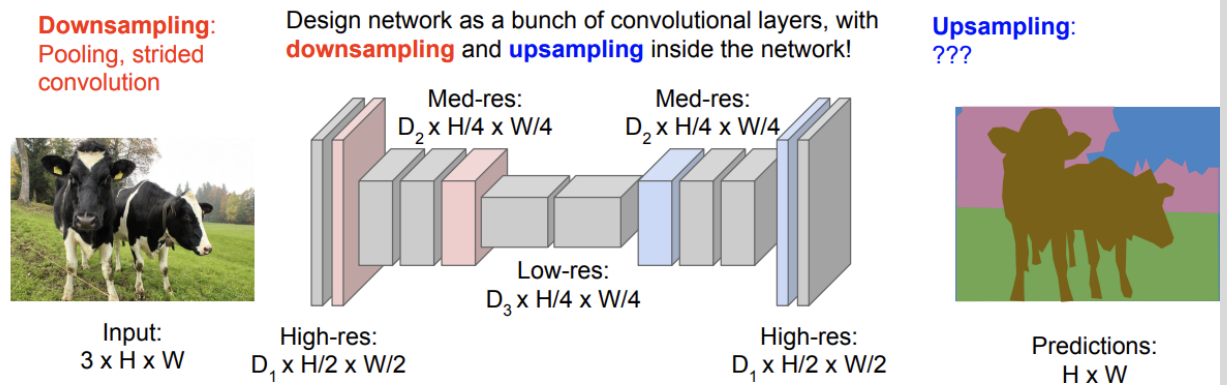
원본 이미지와 크기를 같게 유지해야 하는게 어렵다

Semantic Segmentation Idea: Fully Convolutional



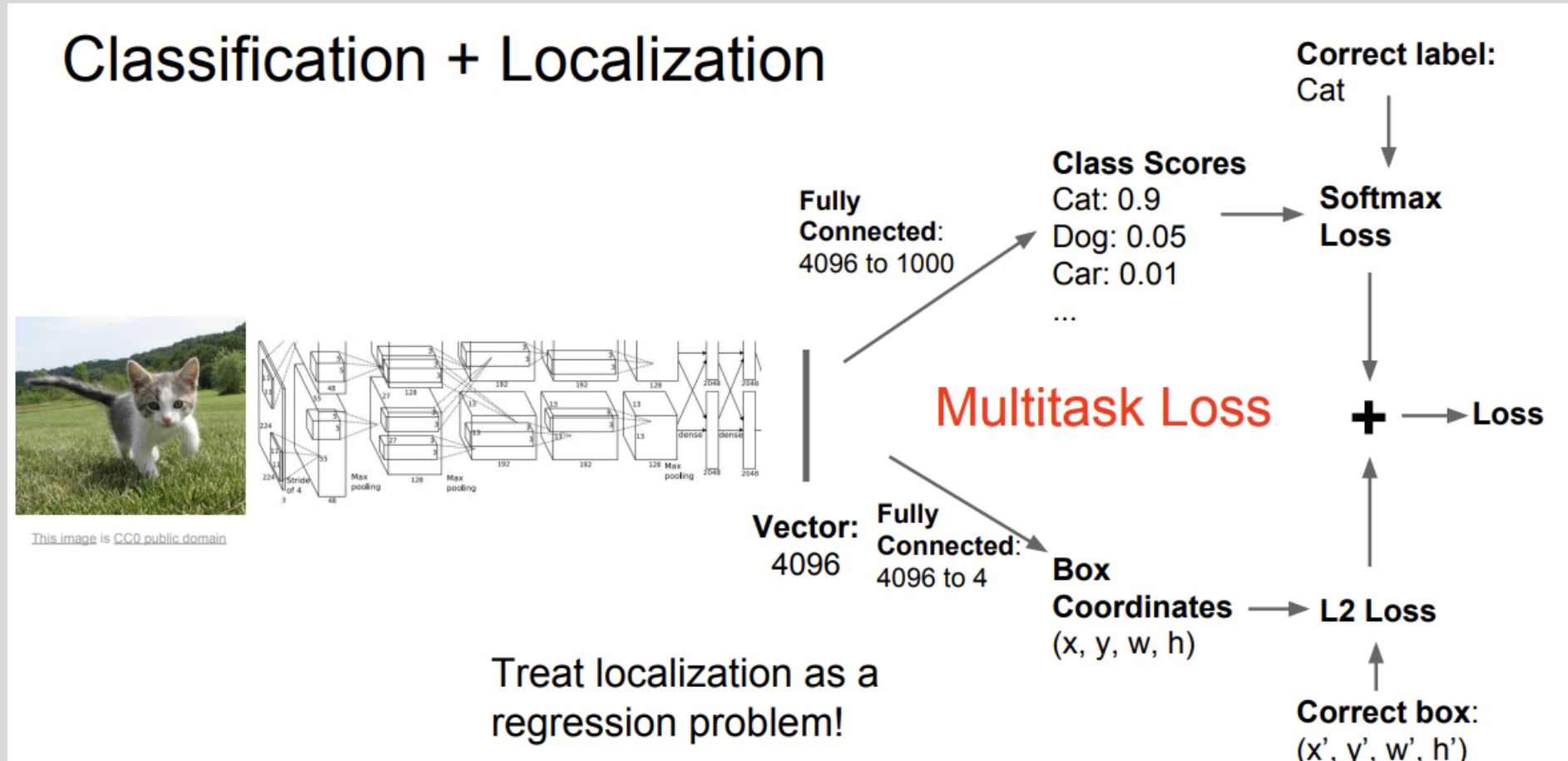
Upsampling과 downsampling을 통해 이를 해결한 방법

Semantic Segmentation Idea: Fully Convolutional



Classification + Localization

물체를 인식하고 위치를 파악한다. 이를 위해 물체를 인식하는 것의 loss와 물체의 위치를 파악한 것의 loss를 둘 다 사용한다.

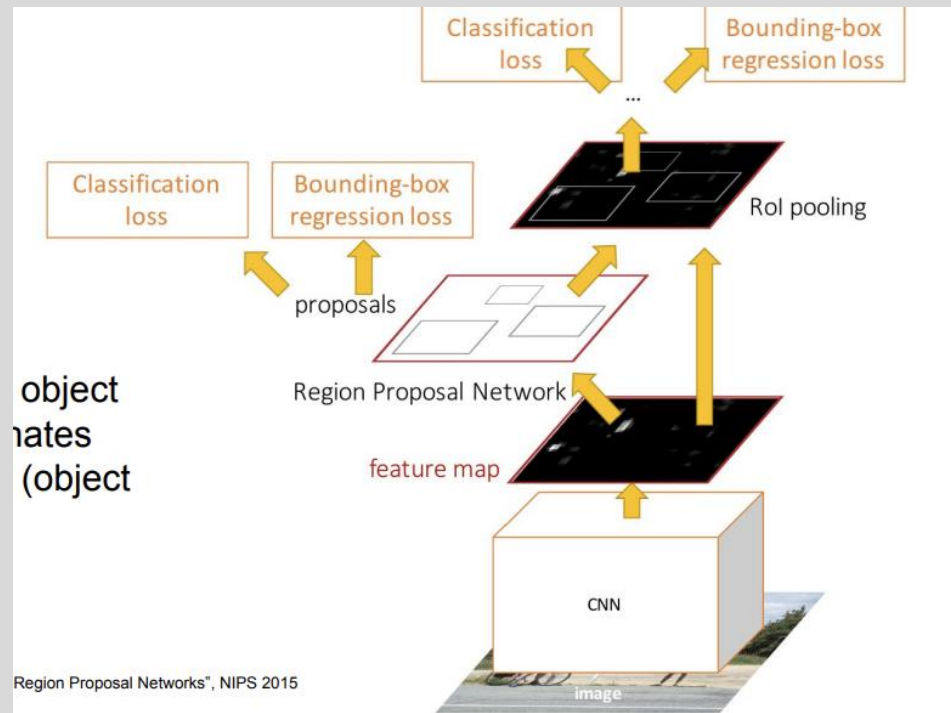


Object Detection

이미지에서 여러 물체를 인식하고 이것들의 위치를 파악하는 작업이다.

단계별로 classification 과 localization을 수행하는 2stage detection 방법과 이를 한번에 수행하는 1stage detection 방법이 있다.

우선 픽셀을 통해 물체가 위치했을 것 같은 예상 구역을 주고 그것들에 대한 classification과 localization을 진행한다. 이를 통한 방법에는 R-CNN, Fast R-CNN, Faster R-CNN이 있다.



이미지에서 물체의 예상위치를 선정하는 것이 Fast R-cnn에서의 병목과정인데 Faster R-CNN에서는 예상위치를 제시하는 네트워크를 이미지를 CNN모델에 적용 후 위치시켜 예상위치를 주면 원본 이미지에 적용하였던 CNN모델을 이용하여 물체 classification과 localizaiton을 수행하는 것이다.

1 stage detection 에는 YOLO와 SSD라는 모델이 있고 이들은 물체의 위치를 예상하면서 confidence라는 변수를 통해 그것이 어떤 class에 속하는지에 대해서도 예측한다

ConvNets의 원리

Convolution 네트워크에서 첫번째 레이어의 필터들은 이미지의 선과 같은 직관적으로 이해가 가능한 특징들을 추출하지만 중간층에서는 직관적으로 파악할 수 있는 부분이 적어진다.

Weights:



Weights:



Weights:



Convolution 네트워크를 거친 다음 Nearest neighbor를 통해 구한 이미지들을 보면 이전 강의에서 이미지의 픽셀값을 기준으로 Nearest neighbor를 구한 것과는 다르게 이미지와 특징적으로 비슷한 것을 잘 골라내는 것을 알 수 있다.

Test image L2 Nearest neighbors in feature space

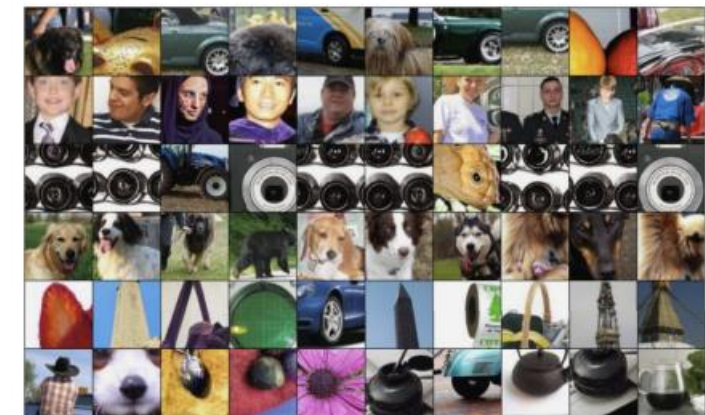
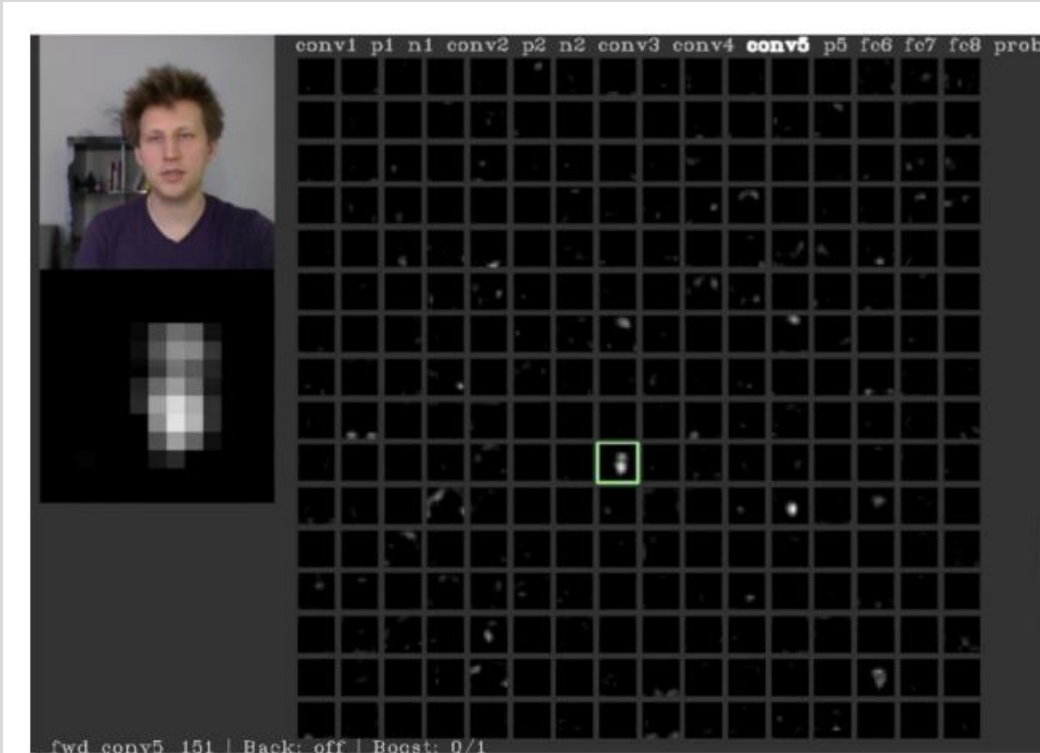


Neural Networks*, NIPS 2012.

ConvNets의 원리

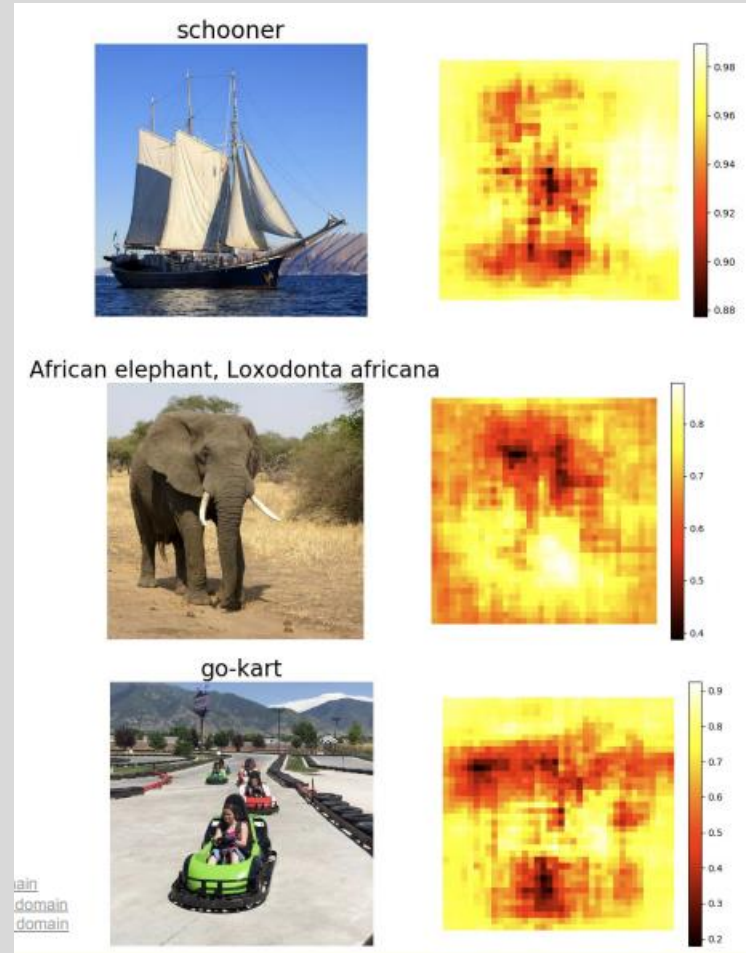
CONVnets에서 특정 이미지에 대해서는 일부의 피쳐 맵에서만 반응하는 것을 확인 할 수 있었다.

이를 확인하고 어떠한 이미지가 각각의 피쳐맵을 가장 활성화 하는지 확인 해볼 수 있다. 확인해 보았을 때 각각의 피쳐맵은 이미지의 일부의 특정 패턴을 통해 활성화되는 것을 알 수 있었다.



ConvNets의 원리

Occlusion Experiment는 이미지의 일부를 가렸을 경우 바뀌는 이미지 예측확률을 보여주는 히트맵이고, 만약 변화가 많다면 가려진 이미지가 물체를 인식하는데 중요한 역할을 한다고 파악할 수 있다.

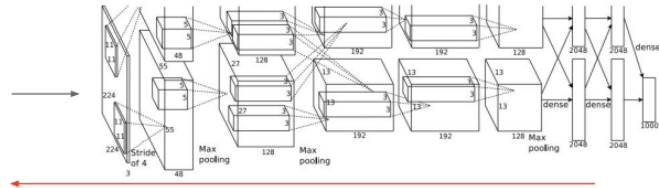


이와 비슷하게 픽셀에 따른 영향을 확인 해보면 heatmap은 물체의 형상과 비슷한 모양을 띄게 된다.

ConvNets의 원리

Gradient Ascent는 필터를 고정시켜 zero이미지를 주어줬을 때 기울기가 증가시키는 방향으로 학습시켜 cnn을 활성화 시키는 방향으로 zero이미지를 변형시키는 방법이다.

1. Initialize image to zeros

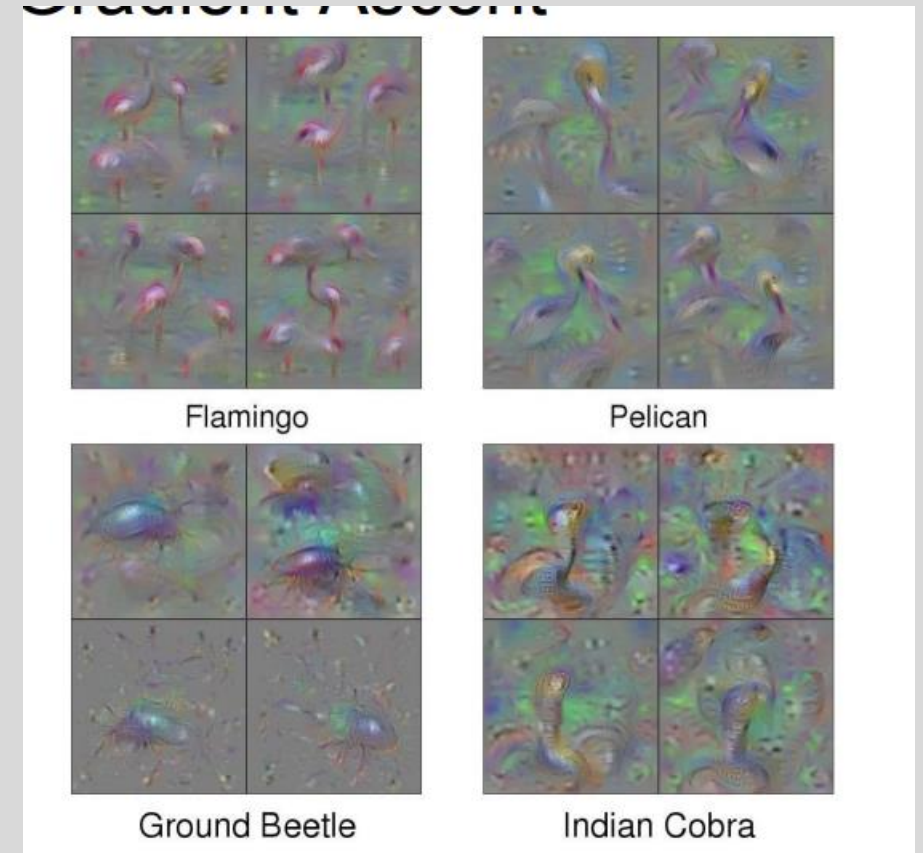


$$\arg \max_I \boxed{S_c(I)} - \lambda \|I\|_2^2$$

score for class c (before Softmax)

Repeat:

2. Forward image to compute current scores
3. Backprop to get gradient of neuron value with respect to image pixels
4. Make a small update to the image



ConvNets 구현

CNN forward

```
pad = conv_param['pad']
stride = conv_param['stride']
N = x.shape[0]
F = w.shape[0]
HH = w.shape[2]
WW = w.shape[3]
H_out = ((x.shape[2]+2*pad-HH)//stride)+1
W_out = ((x.shape[3]+2*pad-WW)//stride)+1

padded_x = np.pad(x, [(0,0), (0,0), (pad,pad), (pad,pad)])
out = np.zeros((N,F,H_out,W_out))

for n in range(N):
    for f in range(F):
        for i in range(H_out):
            for j in range(W_out):
                h_st = i * stride
                w_st = j * stride
                out[n][f][i][j] = np.sum(padded_x[n,:,h_st:h_st+HH, w_st:w_st+WW] * w[f,:,:,:]) + b[f]
```

CNN backward

```
# ***** YOUR CODE (DO NOT DELETE ANYTHING) *****
x, w, b, conv_param = cache

pad = conv_param['pad']
stride = conv_param['stride']

N = x.shape[0]
F = w.shape[0]
HH = w.shape[2]
WW = w.shape[3]
H_out = 1 + (x.shape[2] + 2 * pad - HH) // stride
W_out = 1 + (x.shape[3] + 2 * pad - WW) // stride

x_pad = np.pad(x, [(0,0), (0,0), (pad,pad), (pad,pad)])
dw = np.zeros_like(w)
dx = np.zeros_like(x_pad)
db = np.zeros_like(b)

for n in range(N):
    for f in range(F):
        for i in range(H_out):
            for j in range(W_out):
                h_start = i * stride
                w_start = j * stride
                db[f] += dout[n][f][i][j]
                dw[f,:,:,:] += dout[n][f][i][j] * x_pad[n,:,h_start:h_start+HH, w_start:w_start+WW]
                dx[n,:,h_start:h_start+HH, w_start:w_start+WW] += dout[n][f][i][j] * w[f,:,:,:]

dx = dx[:, :, pad:-pad, pad:-pad]
```

ConvNets 구현

MaxPooling forward

```
ph = pool_param['pool_height']
pw = pool_param['pool_width']
stride = pool_param['stride']

h_out = 1+(x.shape[2]-ph)//stride
w_out = 1+(x.shape[3]-pw)//stride

N = x.shape[0]
C = x.shape[1]
out = np.zeros([N,C,h_out,w_out])

for n in range(N):
    for c in range(C):
        for i in range(h_out):
            for j in range(w_out):
                h_start = i * stride
                w_start = j * stride
                out[n][c][i][j] = np.max(x[n][c][h_start:h_start+ph,w_start:w_start+pw])

# ===== END OF YOUR CODE (DO NOT DELETE/DELETE THIS LINE) =====
```

MaxPooling backward

```
x, pool_param = cache
N, C, H, W = dout.shape
dx = np.zeros_like(x)

s1 = s2 = pool_param['stride']
hp = pool_param['pool_height']
wp = pool_param['pool_width']

for i in range(H):
    for j in range(W):
        [ns, cs], h, w = np.indices((N,C)), i*s1,j*s2
        f = x[:, :, h:h+hp,w:w+wp].reshape(N,C, -1)
        k, l = np.unravel_index(np.argmax(f,2),(hp,wp))
        dx[ns,cs, h+k, w+l] += dout[ns, cs, i, j]
```

다음 주제: Visualizing and Generative Models